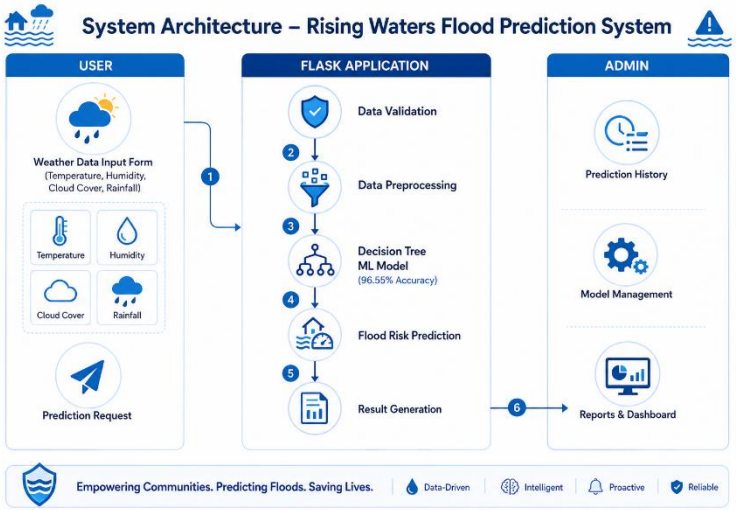
Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	2 July 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode



Guidelines:

- Include all the application processes (User Input, Data Preprocessing, Prediction, Result).
- Show the infrastructure separation (Local / Cloud, if applicable).
- Indicate external interfaces (Weather API, Email/SMS API – optional).
- Include data storage components (Historical Dataset, Trained Model, Prediction Logs).
- Show the connection between the Flask application and the Machine Learning model (Decision Tree).
- Display the flow from user input to flood prediction results.
- Clearly label each technology block (Flask, Decision Tree Model, Dataset, User Interface).
- Keep the architecture simple, clear, and easy to understand.

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Web application for entering weather data and viewing flood prediction results	HTML5, CSS3, JavaScript, Bootstrap 5, Flask Templates
2.	Application Logic-1	Handle user requests, input validation, routing, and prediction workflow	Python, Flask
3.	Application Logic-2	Data preprocessing and feature engineering before prediction	Pandas, NumPy, Scikit-learn
4.	Application Logic-3	Load the trained model and generate flood predictions	Joblib, Decision Tree Classifier (Scikit-learn)
5.	Database	Store historical weather and flood datasets (if required)	CSV/Excel Dataset (or SQLite for logs)
6.	Cloud Database	Cloud database for storing prediction history (Future Enhancement)	IBM Cloudant / IBM Db2 (Optional)
7.	File Storage	Store trained model, datasets, and application files	Local File System (`.pkl`, `.csv`, `.xlsx`)
8.	External API-1	Weather API for real-time weather information (Future Enhancement)	OpenWeatherMap API (Optional)
9.	External API-2	SMS/Email notifications for flood alerts (Future Enhancement)	Twilio API / Email API (Optional)
10.	Machine Learning Model	Predict flood occurrence using historical weather and rainfall data	Decision Tree Classifier (Scikit-learn), Joblib

11.	Infrastructure (Server / Cloud)	Deploy the Flask application locally or on cloud platforms	**Local:** Flask Development Server **Cloud:** IBM Cloud / Render / Railway / AWS (Optional)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Open-source frameworks and libraries used to develop the application	Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, XGBoost, Bootstrap5
2.	Security Implementations	Input validation, secure file handling, error handling, and basic web security practices	Flask Input Validation, HTML Form Validation, Python Exception Handling, OWASP Best Practices
3.	Scalable Architecture	Modular architecture separating User Interface, Application Logic, Machine Learning Model, and Data Storage for future scalability	Flask (3-Tier Architecture), Python, Joblib
4.	Availability	Application can run locally and can be deployed to cloud platforms for continuous availability	Flask Development Server, Render / IBM Cloud / AWS (Future Enhancement)
5.	Performance	Fast prediction using a pre-trained Decision Tree model with efficient data preprocessing and lightweight deployment	Decision Tree Classifier, Scikit-learn, Joblib, Pandas